Serving LLMs Under Concurrent Load on a Single T4: A Performance Engineering Study

Sudarshan Acula Srinivasalu
github.com/sudarshan1x1/efficient-inference

November 2026

Abstract

I built an asynchronous load-testing and profiling framework around vLLM and ran eight experiments on Qwen2.5 models (0.5B to 3B parameters) on one 15 GB NVIDIA T4, at 1 to 100 concurrent users. Every request used a fixed 512-token prompt and a fixed 128-token answer, so configurations differ only in how they serve. At 100 users, continuous batching on an optimized engine delivered 22.6× the output throughput of a naive Hugging Face server (670 vs. 30 tokens/s), decomposing into 2.0× from the engine and 11.3× from batching. Prefix caching added 68% throughput at 50 users. AWQ 4-bit quantization gave 1.86× at one user but 0.97× at 100. A KV cache too small for the load cost only 12% throughput but made steady-state first-token latency 6.6× worse. A two-term bandwidth model (4.6 ms + 4.1 ms per GB of weights per token) fitted across model sizes also predicted the quantized model's speed. The optimizations fall into two kinds: *capacity* optimizations raise the work done per second and cut total latency, while *distribution* optimizations (the chunked-prefill budget, admission control) only decide who waits and when. Once the GPU is saturated, only the first kind reduces latency.

1 Motivation

When several candidates interviewed at the same time, my interview chatbot's replies slowed down, and I could not see why. I had not instrumented the system, so I have no measured numbers for those delays. The chatbot ran conversational technical assessments for an AI recruitment platform. Each live turn generated an adaptive question, checked for repetition and competency coverage, and scored the answer by semantic similarity against an answer key.

The conversational model ran on managed Claude APIs, and self-hosted Sentence-Transformers models handled retrieval and related work. I could redesign the application, but I could not inspect how inference ran. So I changed the workflow: questions were generated once per role and approved by recruiters, the live interview followed a fixed state machine, and final scoring moved to an asynchronous background job.

Those fixes worked, but they treated the model as a black box. This project opens the box: what happens inside a serving engine when many users arrive at once, which bottleneck appears first, and which optimizations actually help. In hindsight, my application fixes used the same ideas serving engines use: reuse repeated work, keep heavy work off the latency-critical path, and bound the work done per turn.

These experiments do not explain my chatbot's delays directly. The managed API ran on serving infrastructure I could not inspect, far larger than one T4. The likelier cause was that each live turn chained several model calls in sequence (question generation, repetition and coverage checks,

scoring), each adding a full round-trip. My redesign took most of those calls off the live turn, which is the principle this report ends on: do less work on the latency-critical path.

2 System and Methodology

The framework drives a model server with simulated users, samples GPU, CPU and engine state every second, and saves every request's timing to Google Drive (Figure 1). It runs end to end in a free Colab notebook. Each experiment is a YAML configuration, and an interrupted run resumes where it stopped.

- **Load generator:** asyncio and aiohttp. C virtual users run a closed loop, each sending its next request when the previous one finishes. A "waves" mode emulates static batching. Every streamed token is timestamped.
- **Servers:** vLLM 0.33.0 behind its OpenAI-compatible API, or a naive Hugging Face transformers server that handles one request at a time behind a lock, like a typical first-version backend.
- **Monitor:** NVML for VRAM and GPU utilization, psutil for CPU and RAM, and vLLM's Prometheus metrics for KV-cache usage, running and waiting requests, preemptions and prefix-cache hits.
- **Runner and analysis:** a configuration matrix in; a summary table plus raw per-request records out; then plots and comparison tables.

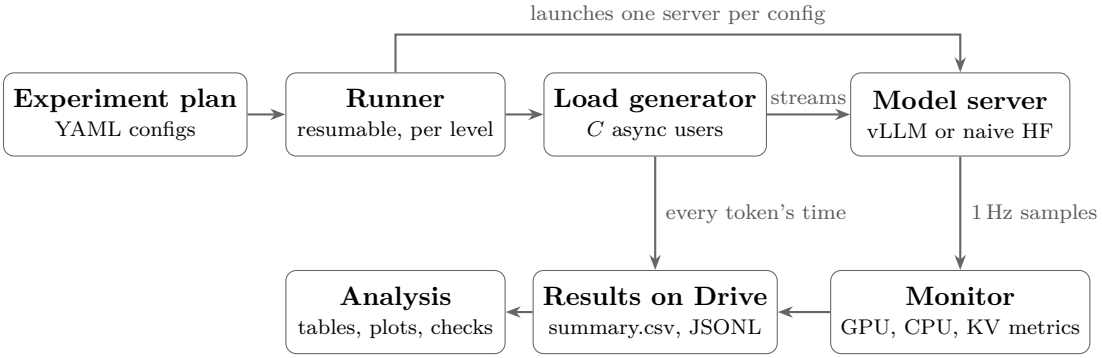


Figure 1: Framework architecture. The runner launches one server per configuration, the load generator streams requests at each concurrency level, and the monitor samples the server and GPU every second.

Hardware and model. One NVIDIA T4 (Turing, sm75, 15 GB, 320 GB/s memory bandwidth, 65 TFLOPS FP16 peak) with 2 vCPUs. Model: Qwen2.5-1.5B-Instruct in FP16 unless stated.

Measurement integrity. Four lessons shaped how results are reported:

1. Each level starts with all C users sending at once. That first cohort dominates TTFT p95, so TTFT is reported for the steady state, excluding it. At 100 users this cut TTFT p95 from 9.4 s to 1.9 s.
2. Latency must count from when work arrives, not when it is sent. Measured from send time, static batching looked 37% better on E2E p95; counted from arrival, it was 19% worse.

Table 1: Workload design.

Choice	Why
Prompt of exactly 512 tokens, sent as token IDs, cut from a real coding corpus (systalyze)	Same input length everywhere; tokenization cannot shift it
Answer of exactly 128 tokens (`min_tokens` = `max_tokens`)	Same decode work per request
Unique prompts, identical across configurations at each load level	Like-for-like comparisons, no accidental cache reuse
Prefix caching off except in Experiment 2	vLLM enables it by default, which would hide inside the baseline
$\max(20, 4C)$ requests per level, after warm-up requests	Stable percentiles; first-call costs excluded

Table 2: Experiments.

#	Experiment	Question
1	Concurrency sweep, 1–100 users	How do engine, batching, KV capacity and quantization change throughput and latency?
2	Prefix caching	What does reusing a shared prompt prefix save?
3	Static vs. continuous batching	What does continuous batching buy when answer lengths vary?
4	KV cache on vs. off (HF)	What does the KV cache itself save?
5	Chunked-prefill budget	How does step size trade first-token time against stalls?
6	Prompt length, vLLM vs. HF	Why is vLLM's first token slower at one user?
7	Model size, 0.5B / 1.5B / 3B	How does decode time scale with weight bytes?
8	Admission control	What does capping concurrent sequences trade?

3. Inter-token gaps are bimodal under load, so p95 can miss the slow cluster entirely. p99 and the share of stalled gaps are reported too.

4. NVML GPU utilization read about 100% for every vLLM run, even at one user, and VRAM stayed flat because vLLM reserves memory at startup. KV-cache usage was the informative memory signal.

The instrumentation was checked against theory throughout. KV usage matched tokens in flight divided by capacity within 0.5 percentage points in five experiments. Prefix-cache hit rates matched $87.5\% \times (n-1)/n$ exactly.

3 Concurrency: Engine and Batching

Without batching, throughput stayed flat at every load and latency was pure queueing. With continuous batching, throughput rose 11.5× from 1 to 100 users (Figure 2).

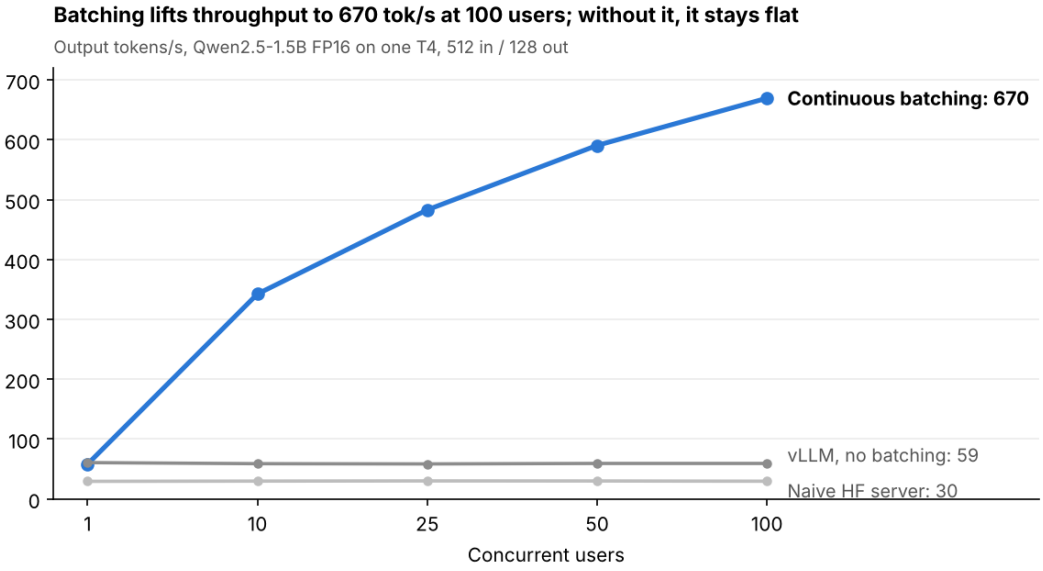


Figure 2: Output throughput vs. concurrent users (Experiment 1).

Table 3: Latency at 100 concurrent users.

Configuration	TTFT p50	E2E p95	ITL p50
Naive HF server	217.6 s	411.4 s	30.4 ms
vLLM, one request at a time	107.1 s	205.2 s	16.0 ms
vLLM, continuous batching	1.4 s (steady state)	26.7 s	67.6 ms

TTFT for the two baselines is queueing time: request 100 waits for the 99 ahead of it, and the queue measured exactly $C-1$ at every level.

The engine alone is worth 2.0×. With batching off in both, a token took 30.4 ms in Hugging Face and 16.0 ms in vLLM. At one user, the GPU's real work per token is about 10 ms: reading 3.1 GB of weights at 320 GB/s. Hugging Face spends the rest launching hundreds of small GPU

kernels from Python, and NVML showed its GPU busy only 52% of the time. vLLM records each decode step once as a CUDA graph and replays it.

Batching is worth 11.3× because decode is memory-bound. Every decode step reads all the weights once, whether it serves 1 sequence or 100. Batching shares that read, so ITL rose only from 16 to 23 ms at 10 users while throughput grew 5.9×.

The gain flattens after about 50 users. Each request carries 512 prompt tokens and 128 answer tokens, so at high load about 80% of processed tokens are prompt tokens. Prompt processing is compute-bound, and batching does not make it free. Total processing reached about 3,350 tokens/s at 100 users.

Continuous beats static batching, but long answers pay for it. In Experiment 3, answer lengths varied from 16 to 256 tokens. Continuous batching delivered 21–29% more throughput (Figure 3), and all requests finished 17–22% sooner counted from arrival. At 50 users, though, the longest 10% of answers took 18.4 s under continuous batching vs. 11.8 s under static waves. Continuous batching refills a freed slot immediately, so every newcomer's prompt processing interrupts long-running answers.

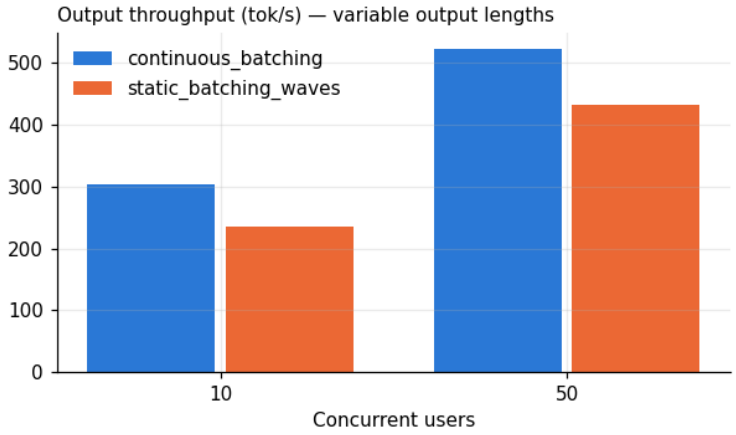


Figure 3: Output throughput, continuous vs. static batching with variable answer lengths (Experiment 3).

4 KV Cache and Memory

The KV cache makes per-token cost flat, but at concurrency its size becomes a hard ceiling on how many users the GPU can serve at once.

Without a cache, every step re-processes the whole sequence. Table 4 shows Experiment 4 (Hugging Face, one request).

At 128 tokens, recomputing everything costs almost nothing extra. The step is dominated by reading the weights, and that read costs the same for 1 token or about 200. Past a few hundred tokens the work turns compute-bound: each extra token cost about 0.16 ms, or roughly 19 TFLOPS achieved (30% of peak).

Table 4: Per-token decode step with and without the KV cache.

Prompt tokens	Cache on	Cache off	Slowdown
128	32.3 ms	39.5 ms	1.2×
512	38.9 ms	103.1 ms	2.7×
1024	32.7 ms	185.7 ms	5.7×

Capacity is set by bytes per token. Qwen2.5-1.5B stores 28 KiB per token (28 layers × 2 for keys and values × 2 KV heads × 128 dimensions × 2 bytes). After weights, vLLM's 8.38 GiB pool held 313,920 tokens. Measured KV usage matched tokens in flight divided by capacity at every level: 10.2% predicted and 10.2% measured at 50 users.

Paging is what makes that pool usable. A server that reserves the full 2,048-token maximum per request fits 153 requests in that pool, and 69% of each reservation goes unused. vLLM allocates 16-token pages as tokens arrive, so 490 requests of 640 tokens fit, wasting at most one page each. Supporting large batches has a cost of its own: limited to one sequence, vLLM kept 18,160 more tokens of KV space, about 0.5 GB that otherwise goes to CUDA graphs and buffers for large batches.

Starving the cache is a cliff, and throughput hides it. With GPU memory capped at 35%, capacity fell to 48,080 tokens, enough for 75 requests of 640 tokens. Up to 50 users it matched the full cache exactly. At 100 users, the running count plateaued at about 75, the rest waited, and vLLM preempted requests 59 times (Figure 4). Throughput fell only 12%, but steady-state TTFT p95 rose from 1.87 s to 12.25 s (6.6×) and E2E p95 by 46%. A preempted request must redo its prompt and queue again.

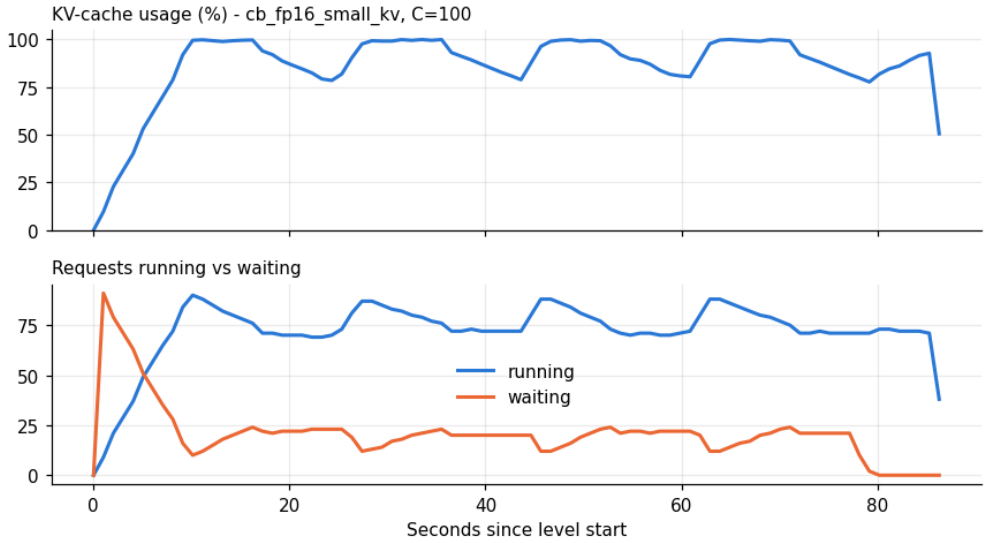


Figure 4: Small KV cache at 100 users: usage hits 100% within about 10 s, the running count plateaus near the 75-request ceiling, and 20–25 requests wait for the rest of the run.

Prefix caching removes repeated prompt work. Experiment 2 used a shared 448-token prefix plus 64 unique tokens (Table 5).

Table 5: Prefix caching off / on.

Users	TTFT p95	Throughput (tokens/s)	KV used
1	118.7 / 58.7 ms	61.8 / 60.1	0.2 / 0.2%
10	1,047 / 321 ms	351 / 436	2.0 / 0.7%
50	4,423 / 1,084 ms	589 / 992	10.1 / 3.1%

Hit rates were 83.1%, 85.3% and 87.1%, exactly $87.5\% \times (n-1)/n$: only each level's first request computed the prefix. Memory matched the sharing model too: one prefix plus $50 \times (64 + 128)$ unique tokens is 3.2% of capacity, against 3.1% measured. Throughput rose 68% at 50 users with identical answers, because the GPU had been spending most of its time on prompts.

5 Generation Stalls: Prompt Processing Interrupts Answers

Under load, a newcomer's prompt is processed inside the same GPU step as everyone else's next token, and every active answer pauses until that step ends. At 100 users, 16.4% of token gaps were stalls of about 450 ms; the rest took about 68 ms (Figure 5).

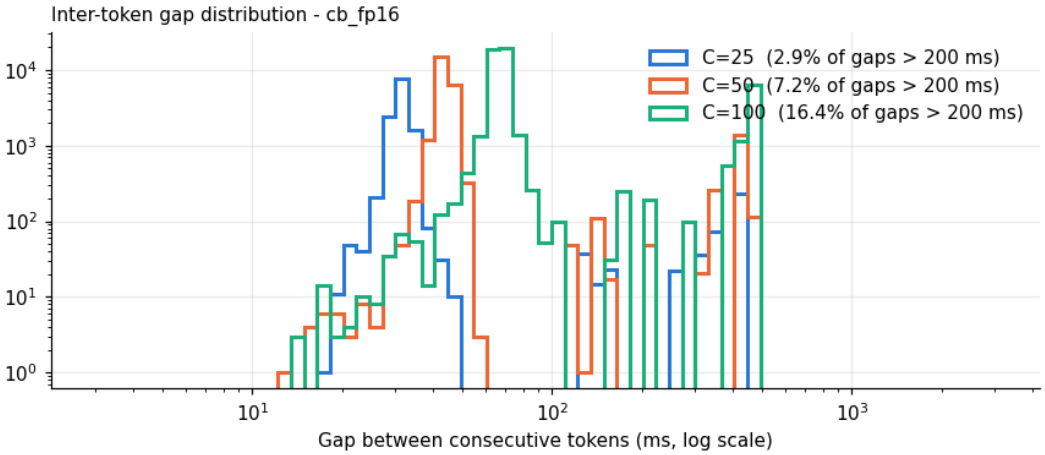


Figure 5: Token gaps under continuous batching: a main cluster of normal steps plus separate clusters of stalls between about 130 and 500 ms.

The stall rate can be predicted from first principles. vLLM processes up to 2,048 tokens per step, about four 512-token prompts, and a full step took about 450 ms. At 100 users, 5.23 requests/s brought 2,678 prompt tokens/s: about 1.3 full prompt steps per second, using 0.59 s of each second. The remaining 0.41 s fits about six normal 68 ms steps, so 1.3 of every 7.4 steps should stall: 18% predicted, 16.4% measured. Three checks confirmed the cause:

1. **Removing prompt work removed the stalls.** With prefix caching, each newcomer computed 64 tokens instead of 512, and ITL p95 at 50 users fell from 429 ms to 48 ms.
2. **CPU was ruled out.** It ran at about 30% throughout the vLLM runs.
3. **Stall sizes appear to come in steps of one prompt.** The slow gaps cluster near 130, 250, 350 and 450 ms, one level for each of 1 to 4 prompts per step.

The step budget trades stall size against stall frequency. Experiment 5 varied vLLM's `max_num_batched_tokens` (Table 6).

Table 6: Chunked-prefill budget.

Budget (tokens)	Stall share	Typical stall	ITL p99	TTFT p95
	50 users		100 users	
512	33.8%	123 ms	155 ms	10.98 s
1,024	16.6%	229 ms	258 ms	9.75 s
2,048 (default)	7.9%	437 ms	463 ms	9.37 s
4,096	3.6%	850 ms	874 ms	8.86 s

Each doubling halved how often stalls happened and doubled their length, about 0.2 ms per prompt token. At 100 users with a 512 budget, stalls vanished into a uniformly slower stream: every step held some prompt work, and the median gap rose from 68 to 137 ms.

The average is conserved. Mean token gap at 100 users came to about 140, 130, 131 and 124 ms across the four budgets. Chunked prefill does not remove prompt work; it decides whether answers see a few long pauses, many short ones, or a steady slowdown.

Two measurement traps. At budget 4,096 and 50 users, ITL p95 was 52 ms while p99 was 867 ms: stalls were only 3.6% of gaps, below p95's reach. And fixed answer lengths exaggerate stalls: users finish together, so newcomers arrive in clumps. With variable lengths in Experiment 3, stalls over 300 ms fell to 2.3% of gaps, while 100–300 ms stalls rose to 26.9%.

6 Memory Bandwidth, Quantization and Model Size

Single-user generation time follows a two-part model: a fixed cost per step plus the time to read the weights. Fitted on the 1.5B and 3B models (Experiment 7):

$$t_{\text{token}} \approx 4.6 \text{ ms} + 4.1 \frac{\text{ms}}{\text{GB}} \times \text{weight size}. \tag{1}$$

That slope is an effective bandwidth of about 240 GB/s, 76% of the T4's peak.

Table 7: Model size (FP16). Bandwidth floor = weights / 320 GB/s. Users that fit = KV capacity / 640 tokens.

Model	Weights	ITL p95	Floor	TTFT p95	Tokens/s	Users
			1 user		100 users	in KV
Qwen2.5-0.5B	0.99 GB	7.8 ms	3.1 ms	54.8 ms	1,795	∼1,490
Qwen2.5-1.5B	3.09 GB	17.3 ms	9.6 ms	120.7 ms	674	∼490
Qwen2.5-3B	6.17 GB	30.0 ms	19.3 ms	201.3 ms	425	∼245

The model predicts the 0.5B result. It gives 8.7 ms against 7.8 ms measured. The 0.5B model has 3.1× fewer weight bytes than the 1.5B but is only 2.2× faster: about 60% of its token time is fixed overhead.

The same model predicts quantization. AWQ stores the main layers of the 1.5B model in 4 bits (about 0.65 GB) and keeps the shared embedding and output layer in FP16 (about 0.47 GB). The model predicts $4.6 + 1.1 \times 4.1 = 9.1$ ms per token; Experiment 1 measured 8.3 ms (p50) to 10.1 ms (p95).

Table 8: AWQ throughput relative to FP16.

Users	1	10	25	50	100
AWQ / FP16	1.86×	1.32×	1.18×	1.06×	0.97×

Quantization helps memory-bound decode, not compute-bound load. At one user, fewer bytes per weight means faster tokens. At 100 users the GPU is compute-bound, converting 4-bit weights back to FP16 becomes overhead, and AWQ loses slightly. TTFT was 2–8% worse with AWQ at every level for the same reason. It did free memory: KV capacity rose 23%, to 384,944 tokens.

Bigger models are slower and fit fewer users. Prompt processing used the GPU better as models grew (9.2, 13.1 and 15.7 TFLOPS at one user), because larger matrix multiplications run more efficiently. But each 3B token needs 36.9 KB of KV cache vs. 28.7 KB for 1.5B, from a smaller pool. At 100 users the 3B model delivered about 4.3 tokens/s per user, below the roughly 5 tokens/s people read at. Output quality was not measured, so this is the cost half of choosing the smallest model that is good enough.

7 A Hardware-Specific Bottleneck: Attention on the T4

vLLM's prompt processing got more expensive per token than Hugging Face's as prompts grew, which points to the attention kernel available for this GPU. My first explanation, fixed per-request overhead, was wrong, and a dedicated experiment showed it.

The prediction. At one user and 512 tokens, vLLM's TTFT (∼125 ms) trailed Hugging Face's (∼69–85 ms). Experiments 2 and 4 suggested about 50 ms of fixed overhead in vLLM, so I predicted a constant gap of about 50 ms at every prompt length.

The result (Experiment 6, Table 9) falsified it.

Table 9: TTFT p95 at one user vs. prompt length.

Prompt tokens	vLLM	HF	Gap
128	43.6 ms	53.1 ms	−9.5 ms
256	61.7 ms	49.3 ms	+12.4 ms
512	125.3 ms	85.2 ms	+40.1 ms
1,024	309.3 ms	186.8 ms	+122.5 ms

At 128 tokens vLLM was faster, so its fixed overhead is small. The gap grew with length: from 512 to 1,024 tokens, each extra token cost vLLM 0.36 ms and Hugging Face 0.20 ms. Cost that grows faster than prompt length points at attention, the only part of the model whose work grows with

the square of the length. Both engines run the weight multiplications through the same NVIDIA libraries. The attention kernels differ: FlashAttention needs a newer GPU (sm80+), so vLLM falls back to a Triton kernel on the T4, while PyTorch's built-in attention has a kernel tuned for this architecture.

A second clue points the same way. vLLM's per-token time at one user rose 11%, from 16.4 to 18.2 ms, as context grew from 128 to 1,024 tokens. Reading the extra cached data explains only about 0.1 ms of that.

This is a strong hypothesis rather than a proof, since I did not profile individual kernels. The same software hits a different bottleneck on different silicon: on an A100 or H100, FlashAttention would likely erase this gap.

Prompt length also sets output capacity. At 25 users, output throughput fell from 915 tokens/s at 128-token prompts to 267 tokens/s at 1,024. Total processing, prompt plus output, levelled off at about 2,400 tokens/s. Past that ceiling, every extra prompt token directly costs output, so longer retrieved context in a RAG system means fewer users served.

8 Admission Control and Little's Law

Capping how many requests share the GPU did not reduce total waiting; it only moved the wait between the first token and the stream that follows (Experiment 8, 100 users, Table 10).

Table 10: Admission control at 100 users.

Max sequences	TTFT p95	ITL p95	E2E p95	Tokens/s	Requests/s
16	26.6 s	27 ms	30.7 s	458	3.57
32	20.7 s	328 ms	27.9 s	536	4.19
64	14.5 s	436 ms	26.3 s	624	4.88
256	9.1 s	456 ms	25.9 s	677	5.29

With a cap of 16, users waited about 26 s for a first token and then streamed smoothly. With a cap of 256, the first token came at 9 s, followed by a slower stream with stalls. Either way, the full answer arrived at about the same time. That is Little's law: with $L = 100$ users always in the system, mean latency W is set by throughput λ,

$$W = \frac{L}{\lambda} : \qquad \frac{100}{3.57 \text{ req/s}} \approx 28 \text{ s}, \qquad \frac{100}{5.29 \text{ req/s}} \approx 19 \text{ s}. \tag{2}$$

Only throughput moves total latency. A low cap cost 32% of it, because fewer sequences shared each read of the weights. KV usage tracked the cap exactly: 3.3% predicted and 3.1% measured at 16.

The cap of 16 showed no stalls for a reason specific to this test. With fixed answer lengths, the 16 admitted users finished together and the next 16 were admitted together, so their prompts were processed while no one was generating. With varied lengths, most of that smoothness would likely disappear.

Admission control still has two real uses. It protects KV memory from the preemption cliff shown earlier, and under overload, rejecting a request quickly can beat letting every user degrade at once. It cannot create capacity.

9 Engineering Conclusions

Once the GPU is saturated, scheduling settings only choose who waits and when. To lower latency, do less work or do the same work more efficiently.

Table 11: Capacity vs. distribution optimizations.

Optimization	Kind	Measured effect	Why
Continuous batching	Capacity	11.3× throughput, 100 users	Decode is memory-bound; a batch shares each weight read
Optimized engine	Capacity	2.0×, one request at a time	CUDA graphs remove Python and kernel-launch overhead
Prefix caching	Capacity	+68% throughput, 50 users	Skips repeated prompt computation
AWQ 4-bit weights	Capacity (low load)	1.86× at 1 user, 0.97× at 100	Fewer bytes help memory-bound decode, not compute-bound load
Smaller model	Capacity	2.7×, 1.5B to 0.5B, 100 users	Fewer bytes and FLOPs per token, down to a fixed-overhead floor
Chunked-prefill budget	Distribution	Stall size vs. frequency; mean gap unchanged	Same prompt work, sliced differently
Admission control	Distribution	First-token wait vs. smoothness; E2E unchanged	Little's law: latency = users / throughput

What helped, what did not, and why.

- Batching and the engine delivered almost all of the gain, because single-user decode wastes a memory-bound GPU.
- Prefix caching was the largest gain once batching was in place, because at high load most of the GPU's work was prompts.
- Quantization did not help at high load on this GPU. It is a memory and low-load optimization, not a throughput fix for saturation.
- Throughput alone hid the worst failure. A KV cache that cost 12% of throughput was 6.6× worse on steady-state first-token time, so KV usage needs its own alert.

For an interview chatbot like mine.

1. Put fixed content (system prompt, rubric, job description, earlier turns) first and per-candidate content last. Prefix caching only matches identical opening tokens.
2. Set the prefill budget for peak load, not average load. On this hardware, 1,024 tokens kept stalls near 230 ms at 50 users; 512 kept every gap under about 155 ms at 100.
3. Watch KV-cache usage, not only throughput, and cap concurrency below the point where preemption begins.
4. Pick the smallest model that meets the quality bar: on one T4, the 3B model fell below reading speed at 100 concurrent streams, and the 1.5B did not.
5. Keep heavy, non-interactive work such as final scoring in background batch jobs, where the large budgets that maximize total throughput are acceptable.

10 Limitations and Future Work

The results describe one GPU generation, one model family and a closed-loop load pattern; each limitation below names the experiment that would lift it.

Limitation	Effect on results	Next step
Closed loop with all users starting at once	Inflates first-cohort TTFT; mitigated by steady-state reporting	Open-loop Poisson arrivals at fixed request rates
Fixed answer lengths in most experiments	Synchronized finishes exaggerate stall size and flatter low admission caps	Repeat Experiments 1, 5 and 8 with variable lengths
One T4 (sm75)	Attention-kernel and bandwidth findings are chip-specific	Repeat on an A100 or L4 with FlashAttention
Attention-kernel cause not profiled	Experiment 6 conclusion is a hypothesis	vLLM's FLEX_ATTENTION backend; kernel timing with Nsight Systems
One model family (Qwen2.5)	Bandwidth fit uses three points from one architecture	Add a Llama-family model of similar size
Output quality not measured	Model-size and quantization results are cost-only	Score a fixed evaluation set per model and quantization
Client and server share 2 vCPUs	Possible timing noise; CPU stayed near 30%	Run the load generator on a separate machine
10–400 requests per level	Low-concurrency tail percentiles rest on few samples	Repeat runs and report spread

Two techniques would directly attack the bottlenecks found here. Separating prompt processing from generation onto different workers removes generation stalls at the source, as in the DistServe and Splitwise designs. An FP8 KV cache on newer GPUs would roughly double KV capacity, pushing back the preemption cliff.

A Reproducibility

Everything runs from one Colab notebook on a free T4 runtime, in about 3 hours of GPU time across two sessions.

Item	Value
Code	github.com/sudarshan1x1/efficient-inference
Software	vLLM 0.33.0, PyTorch 2.14.0 (CUDA 13.0), Hugging Face transformers
Attention backend	TRITON_ATTN, chosen by vLLM on sm75
Default server settings	FP16, max length 2,048, 85% GPU memory, up to 256 sequences, prefix caching off
Workload	512-token prompts as token IDs from the systalyze aws-p50p90-v1-mintok corpus; 128-token answers
Outputs	summary.csv per experiment; per-request JSONL with every token gap; 1-second monitor CSVs; each server's launch command and log

The framework borrows two ideas from public benchmark repositories. systalyze/vllm-benchmarks supplied the coding corpus and the fixed-output-length method. Alt-Shivam/vllm-benchmarking-studio supplied the sweep structure over concurrency and request counts.